RoomLedger
shared expenses · fair settlements

PRODUCT REQUIREMENTS · SYSTEM DESIGN · IMPLEMENTATION BLUEPRINT

RoomLedger

Complete Project Blueprint

A mobile-first PWA for shared living expenses, deterministic settlements and Gemini-powered household financial intelligence.

CORE PRINCIPLE

AI can interpret, explain and suggest.
It never decides the financial truth of the ledger.

Ledger first · Expense tracker second · AI application third

12	9	8	16
modules	AI features	phases	sections

Prepared for Sahil Sameer Siddique
Stack React · Node.js · MongoDB / PostgreSQL · Zod · Gemini API
Edition v1.0 · October 2026

ABOUT THIS DOCUMENT

Contents

01 Executive summary . . . 3
02 Problem and product vision . . . 5
03 Product architecture . . . 7
04 Feature specification . . . 9
05 The ledger and settlement engine . . . 12
06 Database design . . . 16
07 Backend architecture . . . 20
08 API design . . . 23
09 Gemini AI layer . . . 27
10 Frontend, mobile and PWA . . . 31
11 Security and privacy . . . 34
12 Testing strategy . . . 36
13 Deployment and operations . . . 38
14 Development roadmap . . . 40
15 Risks and mitigations . . . 42
16 Portfolio and resume value . . . 43
A Appendices . . . 44

How to use this blueprint	
Implementers	Sections 5 (ledger), 6 (database) and 8 (API) are the specification. Build them in the order given in section 14.
Decision makers	Sections 1–3 summarise the product, scope and architecture; section 15 lists risks and open decisions.
AI work	Section 9 defines exactly what Gemini may and may not do; read it before writing any prompt.
Mobile	Section 10 covers the installable PWA approach so the app feels native on your phone.

Added in this edition

Beyond the original blueprint: a PostgreSQL option, PWA build plan and offline policy, reference code for splits, settlements, idempotency and concurrency, a threat model, property-based tests with 14 ledger test vectors, an AI evaluation plan, release checklist, Gantt roadmap, risk register, open decisions, extension ideas and a first-week checklist.

SECTION 01

Executive summary

RoomLedger is a mobile-first web app that lets roommates record shared spending, split it fairly, and always know who owes whom — with every number traceable to a real transaction. Gemini accelerates data entry and explains spending, but the financial truth is computed by deterministic code.

12	9	8	0
functional modules	Gemini features	build phases	floating-point money values

At a glance

Item	Decision
Product	RoomLedger — shared living expenses, intelligent settlements, AI-assisted household finance
Initial users	You and your roommate; data model supports larger households from day one (householdId / userId everywhere)
Platform	Installable PWA (React) — one codebase for desktop and phone; Capacitor wrapper possible later for an APK
Stack	React + Vite + Tailwind · Node.js + Express · MongoDB (Mongoose) or PostgreSQL (Prisma) · Zod · Gemini API
Money model	Integer minor units (paise). Payer, beneficiary and repayment are three separate concepts
AI policy	AI may interpret, explain and suggest. It may never write balances, confirm expenses or mark repayments received
Architecture	Modular monolith with a pure, framework-free ledger core that is unit-tested to death
Delivery	8 phases, roughly 20 weeks at 12–16 hours/week; usable by two people from phase 4

The single most important design decision

RoomLedger is a **financial ledger first**, an **expense tracker second**, and an **AI application third**. Every later decision — schema, API, UI, AI boundaries — follows from that ordering.

Three layers, three responsibilities

Layer	Responsibility	Trust level
1 · Ledger core	Splits, rounding, balances, settlement suggestions, reversals. Pure functions, no I/O.	**Authoritative** — only source of money truth
2 · Product services	Auth, households, expenses, settlements, budgets, analytics, notifications, audit.	Enforces rules, permissions and consistency
3 · Gemini layer	Natural-language entry, receipt reading, insights, Q&A, budget coaching.	**Advisory** — output validated, reviewed by a human

Success metrics

Objective	What success means	How to measure
Accuracy	Balances reconcile with expenses, allocations and repayments	Property test: Σ balances = 0 across 10,000 random ledgers; zero reconciliation bugs in real use
Low friction	A normal expense is entered in a few seconds	Quick-add takes ≤ 3 taps/10 s; track median time-to-save
Transparency	Every balance can be explained from transactions	Every balance links to a ledger breakdown screen
Intelligence	Gemini reduces manual work and clarifies spending	≥ 60% of quick-adds accepted without edits; insights cite source transactions
Reliability	No duplicates, unauthorized access or silent balance changes	Idempotency + authz test suites green; audit event for every financial write

SECTION 02

Problem and product vision

The problem

Two people share an apartment. In a typical month you pay ₹8,000 for groceries; your roommate pays ₹6,000 for electricity, Wi-Fi and supplies. You buy a ₹1,500 item only you use. Your roommate pays ₹700 for something that benefits both of you. You lend ₹1,000 and get ₹500 back in cash. Rent is due next week, somebody forgot to log a purchase, and each of you remembers the totals differently.

The hard part is not recording amounts — it is **maintaining an accurate financial relationship across many transactions** and being able to prove it.

Questions RoomLedger must answer instantly

- How much did our household spend this month?
- How much did I actually pay, and how much of it was my responsibility?
- How much did my roommate pay on my behalf?
- Which personal purchases must not affect our shared balance?
- Who owes whom, and why — down to the individual transactions?
- Which bills are upcoming or overdue?
- Can we afford another purchase within the shared budget?
- What changed since our last settlement?

Product objectives

#	Objective	Principle
1	Eliminate mental arithmetic	The app computes; humans confirm
2	Make fairness visible	Every balance is explainable and reproducible
3	Remove awkward conversations	Neutral record + dispute flow instead of arguments
4	Keep entry effortless	Defaults, quick-add, receipts, natural language
5	Earn trust	Audit trail, idempotency, no silent changes

What RoomLedger is not (v1)

- A bank account, wallet, or system that holds or moves money.
- A replacement for bank statements.
- An AI financial adviser giving investment recommendations.
- A full accounting package for businesses.

> **About UPI**
> UPI payments can be *recorded* or *initiated* through deep links, but a payment only counts as settled when the recipient confirms it inside the app. This avoids building payment infrastructure while keeping the ledger honest.

Personas

Persona	Profile	Needs
The organiser (you)	Logs most expenses, cares about accuracy, enjoys the data	Fast entry, reports, budgets, audit trail, AI insights
The casual roommate	Wants minimal effort and no surprises	One-tap confirm, clear “what do I owe”, notifications, easy dispute
Future household member	Joins a 3–4 person flat	Multi-member splits, simplified settlement plan

Core user stories

ID	As a…	I want to…	So that…	Phase
U1	roommate	create a household and invite my flatmate by link	we share one ledger	2
U2	payer	add a shared expense with an equal or custom split	the cost is divided fairly	3
U3	user	mark a purchase as personal	it never touches the shared balance	3
U4	debtor	see exactly what I owe and why	I trust the number	3–4
U5	debtor	record a repayment	the balance reduces once confirmed	3
U6	creditor	confirm or reject a repayment	no money is credited that I never received	3
U7	user	correct a wrong expense without erasing history	the audit trail stays intact	3
U8	user	type “850 groceries, split equally”	I can log in two seconds	6
U9	user	photograph a receipt and split by item	mixed shopping is split correctly	6
U10	household	set budgets and get alerts	we do not overspend	5, 7

SECTION 03

Product architecture

The product is organised into 12 functional modules. They are **separate responsibilities, not separate services**: ship one backend application with well-separated modules (a modular monolith). You get simple deployment today and clean seams to extract services later if ever needed.

#	Module	Responsibility	Phase
01	Identity & access	Registration, login, sessions, OAuth, device revocation	2
02	Household management	Create home, invite roommates, defaults and rules	2
03	Expense ledger	Shared, personal, custom-split, recurring and reimbursable expenses	3
04	Settlement engine	Balances, repayments, confirmations, reconciliation	1, 3
05	Dashboard & analytics	Summaries, trends, category breakdowns, budget views	4–5
06	Bills & budgets	Recurring bills, due dates, thresholds, reminders	5, 7
07	Gemini intelligence	Quick-add, receipts, insights, Q&A, coaching	6
08	Notifications	Expense alerts, disputes, reminders, settlement updates	7
09	Audit & disputes	Change history, review and correction workflows	3
10	Reports & exports	Monthly summaries, CSV, printable statements	5, 7
11	Settings & preferences	Currency, split defaults, privacy, notification prefs	2–5
12	Administration & reliability	Monitoring, scheduled jobs, backups, health	8

System architecture

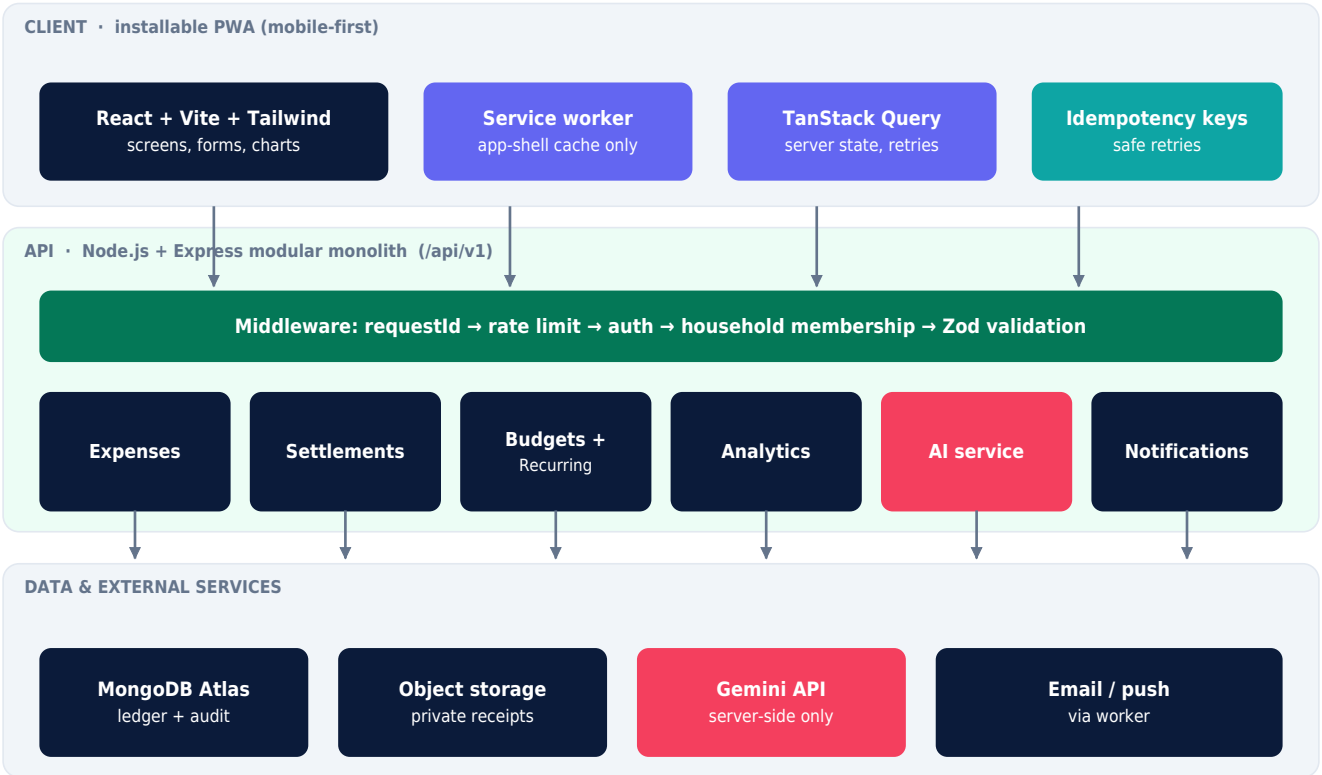


Figure 3.1 — The request path always passes through authentication and household membership checks. The AI service is the only component that talks to Gemini, and it can only reach data through scoped service functions.

Architectural principles

- **Pure core.** Money logic lives in a dependency-free package: no Express, no Mongoose, no clock, no randomness.
- **Thin controllers.** HTTP concerns only; business rules live in services; queries live in repositories.
- **Server is the source of truth.** Clients never submit balances; the server recomputes from records.
- **Append-only mindset.** Corrections are reversals plus replacements; history is never erased.
- **Tenant isolation by construction.** Every query is scoped by householdId derived from verified membership.
- **AI behind a fence.** Gemini output is a draft: validated, previewed, and saved through the normal service.

Technology choices and rationale

Concern	Choice	Why
Frontend	React + Vite + Tailwind CSS	You already ship React; Vite gives fast builds and first-class PWA plugin
Server state	TanStack Query	Caching, retries, optimistic updates, offline-aware mutations
Forms	react-hook-form + Zod resolver	Same Zod schemas validate client and server
Charts	Recharts (or Chart.js)	Accessible, composable, small enough for code-split pages
Backend	Node.js + Express	Matches your resume stack; mature middleware ecosystem
Validation	Zod	Request bodies, env config and Gemini outputs validated with one library
Database	MongoDB (Mongoose) *or* PostgreSQL (Prisma)	See comparison below; decide before phase 1
Cache / limits	Redis (already in your toolbox)	Rate limiting, token blacklist, idempotency key store, job locks
AI	Google Gen AI SDK for Node.js	Structured JSON output, multimodal receipts, function calling
Testing	Vitest/Jest, Supertest, fast-check, Playwright	Unit, API, property-based invariants and end-to-end flows

Database decision: MongoDB vs PostgreSQL

Criterion	MongoDB + Mongoose	PostgreSQL + Prisma
Fit with ledger data	Works; relationships enforced in code	**Natural** — foreign keys, CHECK constraints, joins
Transactions	Multi-document transactions need a replica set (Atlas provides it)	First-class ACID transactions
Integrity guarantees	Application-level (e.g. split sum = amount)	Can add DB-level constraints and deferred triggers
Aggregations / analytics	Aggregation pipeline, flexible	SQL GROUP BY / window functions, very readable
Your experience	PrepStack	SkillBridge, VaultDrive (Neon + Prisma)
Blueprint default	Schemas below are written for MongoDB	Maps 1:1 to tables; collections → tables

Recommendation: PostgreSQL if you want the strongest integrity story for a financial app; MongoDB if you want maximum speed from familiarity. Either is valid — pick one, never both. The rest of this document uses collection names that translate directly to table names.

SECTION 04

Feature specification

4.1 Authentication and account management

Each roommate has an individual account. Shared logins destroy attribution and make audit history meaningless.

Feature	Details
Registration & login	Email + password, optional Google OAuth 2.0 with account linking, email verification
Sessions	Short-lived access token + rotating refresh token in HTTP-only, Secure, SameSite cookies; revocation list in Redis
Account recovery	Single-use, expiring password-reset tokens; rate-limited
Profile	Name, avatar, notification and display preferences
Devices	List active sessions; revoke one or all; password change revokes all
Data rights	Export my data; delete my account with defined effect on households and history

Business rules

- Every operation is performed by an authenticated user.
- Every household resource is checked against active membership — knowing an expense ID never grants access.
- Private expenses are visible only to their owner (policy enforced server-side, not in the UI).
- A user cannot add themselves to a household without a valid, unexpired, single-use invitation.

4.2 Household management

The household is the tenant boundary. Features: create a household (e.g. “Flat 302”), invite by email or single-use link, accept/reject invitations, set currency, timezone and default split, view members and activity history, configure budgets, leave or transfer ownership through a defined workflow.

> **Leaving a household with a non-zero balance**
> Block the leave action (or require a settlement/waiver record) while balances are outstanding. Historical expenses remain; the member status becomes *left* with a leftAt timestamp.

4.3 Expense management

Every expense answers: what, how much, who paid, who bears the cost, which household, when, which category, is there a receipt or note, and does it need review or settlement.

Type	Example	Who bears the cost?	Shared ledger?
Shared	Groceries for both	Both, per split	Yes
Personal	Your headphones	You alone	No (personal analytics only)
Paid on behalf	Medicine for roommate	Roommate (if agreed)	Yes
Unequal shared	Appliance used 70/30	Custom allocation	Yes
Household bill	Electricity, Wi-Fi, rent	Per household rule	Yes
Refund / correction	Store refunds an item	Original beneficiaries	Yes (negative effect)

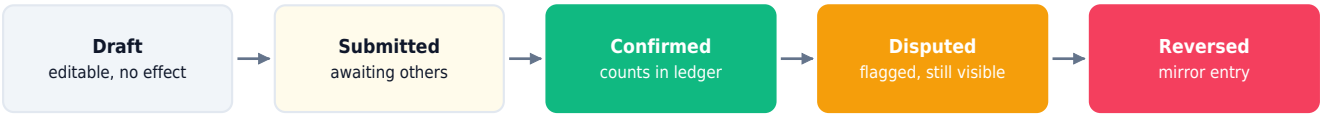
Expense form fields

Description · amount · date/time · payer · expense type · category · split method · beneficiaries and per-person allocation · payment method (cash, UPI, card, bank transfer, other) · merchant · notes · receipt attachment · optional recurring rule · optional tags.

Split methods

Method	Example	Implementation note
Equal	₹1,000 between two → ₹500 each	Floor division; distribute leftover paise deterministically
Exact amounts	₹700 / ₹300	Sum of shares must equal total or reject
Percentage	60 / 40	Use basis points (6000 / 4000) and largest-remainder rounding
Selected participants	Roommate not involved	Only listed members receive a share
Shares / weights (extra)	2 : 1 : 1 in a 3-person flat	Generalises equal and percentage; same remainder rule

Expense lifecycle



For two roommates, allow **instant confirmation** by the creator with a dispute option for the other person. Keep the full approval flow as a per-household setting for later. **Confirmation is not reimbursement**: an expense can be confirmed while the payer is still owed money.

4.4 Quick entry and defaults

Quick entry is a compact form (description, amount, payer, split) that applies sensible defaults — payer = me, split = household default, date = now — and shows a review step. Typical inputs:

- “Groceries, ₹1,250, paid by me, split equally.”
- “Electricity bill ₹1,840, paid by roommate, 50/50.”
- “Bought a ₹900 desk lamp for myself.”
- “I paid ₹600 for my roommate’s medicine.”
- “Split ₹2,000 as ₹1,200 for me and ₹800 for my roommate.”

In phase 6, Gemini interprets these sentences (including Hinglish such as “850 ka grocery, half-half”) into the same review form. The result is a draft, never a saved expense.

4.5 Disputes and corrections

1. Roommate taps **Dispute** on an expense and gives a reason; the expense is flagged but still counted (configurable).
2. Creator edits (new version) or reverses and re-enters; both actions write audit events with before/after snapshots.
3. Affected members are notified; the balance recalculates from the corrected records.
4. Unresolved disputes are shown on the dashboard so they cannot be forgotten.

4.6 Notifications

Event	Recipient	Channel (phase)
New shared expense	Other members	In-app (7), push/email later
Expense disputed / corrected	Creator / affected members	In-app
Repayment recorded	Recipient (to confirm)	In-app + push
Repayment confirmed / rejected	Sender	In-app
Invitation received	Invitee	Email
Bill due soon	All members	In-app + push
Budget threshold (e.g. 80%, 100%)	All members	In-app

Notifications are derived from events and are never the authoritative record of a transaction.

4.7 Reports and exports

- Monthly report: totals, paid vs share per member, net balance, top categories, month-over-month change, upcoming bills, budget status.
- CSV export of expenses, settlements and ledger entries; printable PDF statement per period.
- **Close the month** (extra): once both members confirm, snapshot the period so old balances cannot drift silently.

SECTION 05

The ledger and settlement engine

This is where most engineering time belongs. A screen can be redesigned in an afternoon; a settlement engine that miscalculates money destroys trust permanently.

5.1 The fundamental model

For every confirmed shared expense record the **total**, the **payer**, the **people responsible** and the **amount assigned** to each. Record repayments separately. For any member *u*:

Net balance

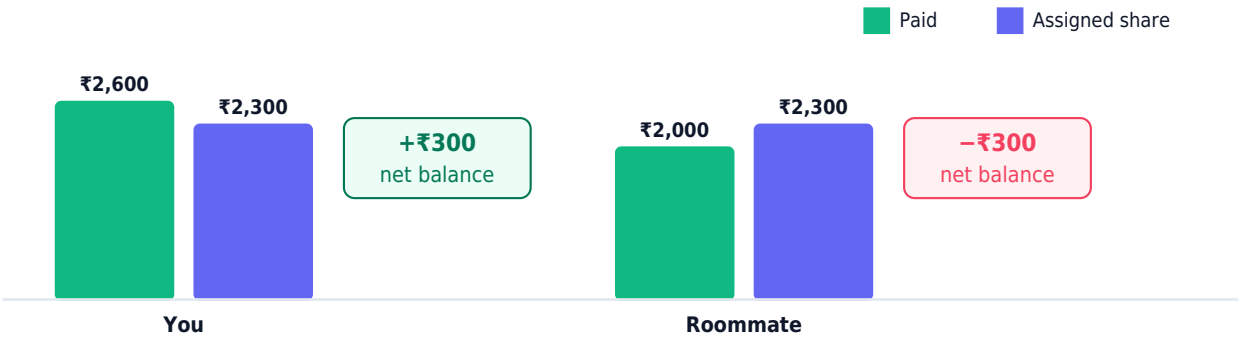
Net(u) = Paid(u) − Share(u) + Sent(u) − Received(u)

Positive → net creditor (is owed money) · Negative → net debtor (owes money) · Zero → settled. Sent/Received count **confirmed** repayments only.

Personal spending is shown separately and is not part of the shared ledger.

5.2 Worked example

Transaction	Payer	Amount	Your share	Roommate share
Groceries	You	₹2,000	₹1,000	₹1,000
Electricity	Roommate	₹1,200	₹600	₹600
Wi-Fi	Roommate	₹800	₹400	₹400
Household item	You	₹600	₹300	₹300
Total		**₹4,600**	**₹2,300**	**₹2,300**



You paid ₹2,600 against a share of ₹2,300 → **+₹300**. Your roommate paid ₹2,000 against ₹2,300 → **−₹300**. Therefore the roommate owes you ₹300. A ₹1,500 personal purchase of headphones changes your personal spending but not this balance.

5.3 Four cases the app must never confuse

Case	Scenario	Ledger effect
A · Shared item	You pay ₹900 for a purifier split equally	Paid +900 (you); shares 450 / 450 → roommate owes you 450 more
B · Personal item	You buy a ₹900 item for yourself	Personal analytics only; no balance change
C · On behalf	You pay ₹900 for roommate's agreed purchase (100% share)	Roommate owes you 900
D · Repayment	Roommate sends you ₹500	Separate settlement record; reduces debt by 500 once confirmed

The crucial rule

Payer, beneficiaries and repayment are **separate concepts**. A person can pay without being responsible for the whole cost, and someone can owe money without making any purchase.

5.4 Reference implementation: split engine

All arithmetic uses integer paise. Percentages are expressed in basis points (1% = 100 bps) to avoid floating-point entirely. Leftover paise go to the participants with the largest fractional remainder, ties broken by userId so results are reproducible.

server/src/modules/ledger/split.js

```
// ledger/split.js  -- pure, deterministic, no I/O
export function splitEqual(totalMinor, userIds) {
  const ids = [...userIds].sort();
  const base = Math.floor(totalMinor / ids.length);
  let left = totalMinor - base * ids.length;           // 0 <= left < n
  return ids.map((userId) => ({ userId, shareMinor: base + (left-- > 0 ? 1 : 0) }));
}

export function splitByBasisPoints(totalMinor, parts) {  // parts: [{ userId, bps }]
  if (parts.reduce((s, p) => s + p.bps, 0) !== 10000) throw new Error('BPS_SUM_INVALID');
  const rows = parts.map(({ userId, bps }) => {
    const num = totalMinor * bps;                      // safe integer for realistic amounts
    return { userId, share: Math.floor(num / 10000), frac: num % 10000 };
  });
  let left = totalMinor - rows.reduce((s, r) => s + r.share, 0);
  rows.sort((a, b) => b.frac - a.frac || a.userId.localeCompare(b.userId));
  rows.forEach((r) => { if (left > 0) { r.share += 1; left -= 1; } });
  return rows.map(({ userId, share }) => ({ userId, shareMinor: share }));
}
```

5.5 Reference implementation: balances and settlement plan

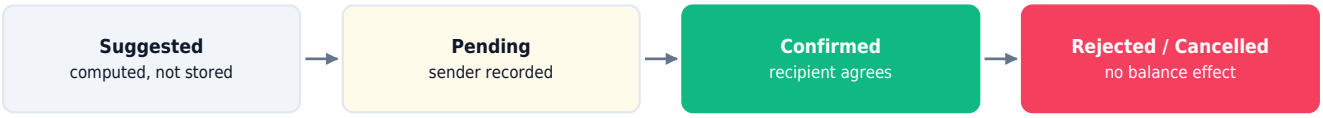
server/src/modules/ledger/balances.js

```
// ledger/balances.js
export function calculateMemberBalances({ expenses, settlements, memberIds }) {
  const net = Object.fromEntries(memberIds.map((id) => [id, 0]));
  for (const e of expenses) {                    // confirmed + shared only
    net[e.paidBy] += e.amountMinor;              // credit the payer in full
    for (const s of e.splits) net[s.userId] -= s.shareMinor;   // debit each beneficiary
  }
  for (const t of settlements) {                 // confirmed only
    net[t.fromUserId] += t.amountMinor;          // sender's debt shrinks
    net[t.toUserId]   -= t.amountMinor;          // recipient's credit shrinks
  }
  const total = Object.values(net).reduce((a, b) => a + b, 0);
  if (total !== 0) throw new Error('LEDGER_NOT_BALANCED');   // invariant: sum is zero
  return net;
}

// ledger/suggest.js  -- greedy matching of largest debtor with largest creditor
export function suggestSettlements(net) {
  const cr = [], db = [];
  for (const [id, v] of Object.entries(net)) {
    if (v > 0) cr.push({ id, amt: v }); else if (v < 0) db.push({ id, amt: -v });
  }
  const order = (a, b) => b.amt - a.amt || a.id.localeCompare(b.id);
  cr.sort(order); db.sort(order);
  const plan = []; let i = 0, j = 0;
  while (i < db.length && j < cr.length) {
    const pay = Math.min(db[i].amt, cr[j].amt);
    plan.push({ from: db[i].id, to: cr[j].id, amountMinor: pay });
    db[i].amt -= pay; cr[j].amt -= pay;
    if (db[i].amt === 0) i++;
    if (cr[j].amt === 0) j++;
  }
  return plan;                                    // suggestions only; never auto-applied
}
```

The greedy plan is always *valid* (it zeroes every balance) but does not always minimise the number of transfers globally. For households of two to four people this is irrelevant; for larger groups you can later replace it with a subset-sum heuristic behind the same function signature.

5.6 Settlements and repayments



- Sender records the repayment; the **recipient confirms receipt**. A disputed or pending repayment never silently reduces the official balance.
- A record contains household, sender, recipient, amount, method, optional payment reference, initiated/confirmed timestamps, status, note and audit history.
- Partial repayments are supported; overpayment flips the direction of the balance (the recipient now owes the sender).
- Prevent self-settlements; validate that both parties are active members; optionally cap a single repayment at the outstanding debt with an explicit override.

5.7 Edits, reversals and refunds

Operation	Rule
Edit confirmed expense	Optimistic-lock on version; recompute splits; write audit event with before/after; notify affected members
Reverse expense	Create a mirror entry (or mark reversed and exclude it) — history is retained; never hard-delete a confirmed expense
Refund / partial refund	Allocate across original beneficiaries pro rata using the same remainder rule, unless users choose another valid treatment (₹200 refund on a ₹1,000 equal split → ₹100 each)
Draft deletion	Allowed; drafts have no ledger effect
Opening balances (extra)	Model as an explicit, dated adjustment so the sum-zero invariant still holds

5.8 Invariants the system must always satisfy

#	Invariant	Enforced by
I1	For every confirmed expense, Σ shareMinor = amountMinor	Split engine + service check + DB constraint where possible
I2	Σ member net balances = 0	Balance function throws; property tests
I3	All money values are integers (paise); no floats anywhere in the ledger path	Zod int() + lint rule + tests
I4	Personal expenses never appear in shared balances	Ledger input filter + unit tests
I5	Pending/rejected settlements never change balances	Ledger input filter + unit tests
I6	Confirmed financial records are never deleted, only reversed	No DELETE route; repository guard
I7	Same idempotency key + payload → same result, one record	Idempotency middleware + unique index

SECTION 06

Database design

Field names are illustrative; the relationships and constraints are what matter. Collections translate one-to-one to PostgreSQL tables if you choose that path. Every collection that holds household data carries **householdId**.

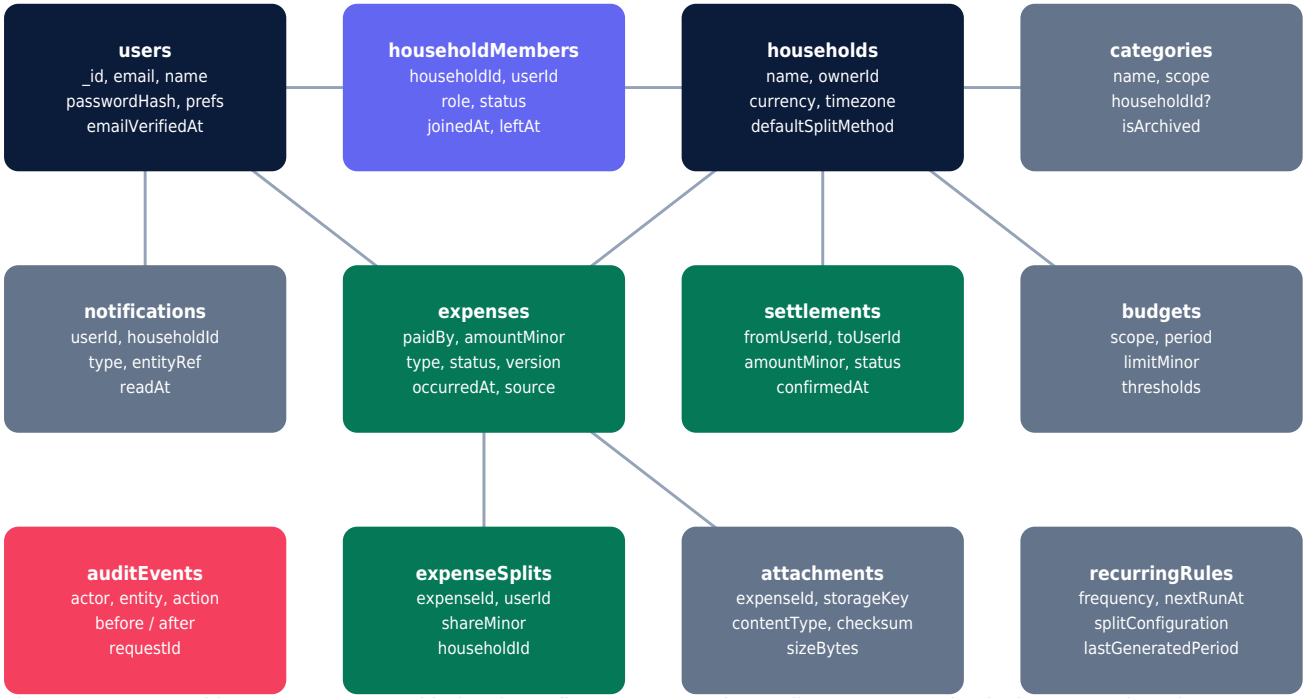


Figure 6.1 — Core entities. Green = money-critical, red = audit, grey = supporting. auditEvents, categories, budgets, recurringRules, attachments, notifications are all scoped by householdId.

6.1 users

Field	Type	Purpose
_id	ObjectId	Primary identifier
name / email	String	Display name; normalised, unique email (case-folded)
passwordHash	String | null	Argon2id or bcrypt hash; null for OAuth-only users; never returned by the API
authProviders	Array	Linked providers (google subject id) for account linking
emailVerifiedAt	Date | null	Verification state
avatarUrl, preferences	String, Object	Profile, locale, theme, notification settings
createdAt / updatedAt	Date	Timestamps

6.2 households and householdMembers

Field	Type	Purpose
households.name, ownerId	String, ObjectId	Display name and current owner
households.currency, timezone	String	Explicit; no mixing currencies without an exchange-rate policy; timezone defines month boundaries
households.defaultSplitMethod, settings	String, Object	Default split, approval and notification policies
households.archivedAt	Date | null	Archival instead of deletion
householdMembers.role / status	String	owner | member · invited | active | left
householdMembers.joinedAt / leftAt	Date | null	Membership window; unique index on (householdId, userId)

6.3 expenses and expenseSplits

Field	Type	Purpose
householdId, createdBy, paidBy	ObjectId	Tenant, author and actual payer
amountMinor	Safe integer / Long	Total in paise; always an integer
currency	String	Household currency code
description, notes, merchant	String	Human details
categoryId	ObjectId | null	Category
expenseType	String	shared | personal | on_behalf | bill | refund
splitMethod	String	equal | exact | percentage | shares
occurredAt	Date	When the spending happened (not when it was entered)
status	String	draft | submitted | confirmed | disputed | reversed
source	String	manual | quick_entry | ai_assisted | recurring
reversalOf, recurringRuleId, idempotencyKey	ObjectId / String	Links for reversals, recurring generation and duplicate protection
version	Number	Optimistic concurrency control
expenseSplits: expenseId, householdId, userId, shareMinor	IDs + integer	One row per beneficiary; Σ shareMinor = expense.amountMinor

6.4 settlements

Field	Type	Purpose
householdId, fromUserId, toUserId	ObjectId	Sender and recipient (must differ; both active members)
amountMinor, currency	Integer, String	Repayment amount
method, paymentReference	String	cash | upi | bank | other; optional reference
status	String	pending | confirmed | rejected | cancelled
initiatedAt, confirmedAt, createdBy, confirmedBy	Date / ObjectId	Who did what and when

6.5 Supporting collections

Collection	Key fields	Notes
categories	name, scope (system|household), householdId, isArchived	Never hard-code names in ledger logic
budgets	scope, userId?, categoryId?, period, limitMinor, startsAt, endsAt, thresholds, isActive	Store rules; compute spent from expenses so refunds are reflected
recurringRules	frequency, interval, nextRunAt, startAt, endAt, splitConfiguration, lastGeneratedPeriod	Generates *expected* expenses; idempotent per period
attachments	expenseId, uploadedBy, storageKey, contentType, sizeBytes, checksum	Files in private object storage; signed URLs
auditEvents	actorUserId, entityType, entityId, action, before/afterSnapshot, reason, requestId	Append-only; no client edit or delete
notifications	userId, householdId, type, title, body, entityRef, readAt	Derived data, not authoritative
invitations	householdId, emailHash/token hash, expiresAt, usedAt, invitedBy	Store only a hash of the token; single use; short expiry
idempotencyKeys	userId, key, requestHash, response, expiresAt	Unique (userId, key); TTL cleanup
aiUsageRecords	user, household, feature, model, tokens, latency, status, estimatedCost	Quota and cost control
aiDrafts	user, household, feature, draftPayload, validation, expiresAt	TTL index; short-lived review drafts

6.6 Essential indexes

Collection	Index	Serves
users	unique(email)	Login, duplicate prevention
householdMembers	unique(householdId, userId); (userId, status)	Membership checks, “my households”
expenses	(householdId, occurredAt desc); (householdId, status, occurredAt desc); unique(userId, idempotencyKey) sparse	Lists, filters, duplicate protection
expenseSplits	(expenseId, userId); (householdId, userId)	Detail view, balance aggregation
settlements	(householdId, status, initiatedAt desc)	Pending confirmations, history
recurringRules	(active, nextRunAt)	Scheduler scan
notifications	(userId, createdAt desc)	Inbox
aiDrafts / idempotencyKeys	TTL on expiresAt	Automatic cleanup

Do not index every field. Add indexes from real query patterns and measure with explain().

6.7 Consistency patterns

server/src/modules/expenses/expense.service.js (excerpt)

```
// Create expense + splits + audit atomically (MongoDB needs a replica set; Atlas has one)
const session = await mongoose.startSession();
try {
  let created;
  await session.withTransaction(async () => {
    const [expense] = await Expense.create([{ ...doc, version: 1 }], { session });
    await ExpenseSplit.insertMany(
      splits.map((s) => ({ ...s, expenseId: expense._id, householdId })), { session });
    await AuditEvent.create([{ householdId, actorUserId, entityType: 'expense',
      entityId: expense._id, action: 'expense.created', afterSnapshot: expense.toObject(), requestId }], { session });
    created = expense;
  });
  return created;
} finally { await session.endSession(); }

// Optimistic concurrency on edit
const r = await Expense.updateOne(
  { _id, householdId, version: expectedVersion, status: { $ne: 'reversed' } },
  { $set: changes, $inc: { version: 1 } }, { session });
if (r.modifiedCount === 0) throw new ConflictError('STALE_VERSION');   // HTTP 409
```

PostgreSQL note

With Postgres add **CHECK (amount_minor > 0)**, **CHECK (from_user_id <> to_user_id)** and a deferred constraint trigger asserting that Σ share_minor = amount_minor per expense. The database then defends the invariants even if application code regresses.

SECTION 07

Backend architecture

7.1 Project structure

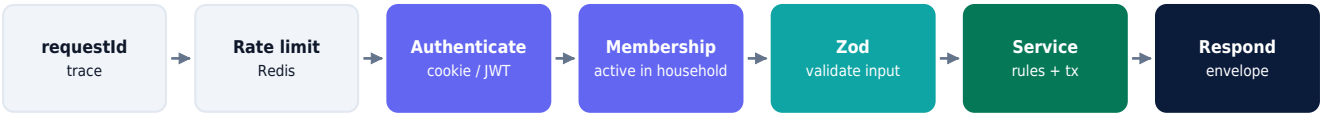
repository layout

```
roomledger/
  client/src/
    app/  components/  hooks/  lib/  services/  pages/
    features/ auth/ household/ expenses/ settlements/ analytics/ budgets/ ai-assistant/
  server/src/
    config/  middleware/  jobs/  shared/  app.js  server.js
    modules/
      auth/ users/ households/ expenses/ ledger/ settlements/
      budgets/ recurring/ analytics/ notifications/ ai/
  packages/ledger-core/        // optional: pure ledger as its own package, zero dependencies
  tests/ unit/ integration/ e2e/
  docs/ PRD.md DATABASE.md API.md SECURITY.md AI_DESIGN.md ADR/
  README.md

modules/expenses/
  expense.routes.js  expense.controller.js  expense.service.js
  expense.repository.js  expense.validation.js  expense.model.js  expense.test.js
```

Layer	Responsibility	Must not
routes	Bind URLs to middleware + controller	Contain logic
controller	HTTP in/out, status codes	Query the database or apply business rules
service	Business rules, permissions, transactions, audit, events	Know about req/res
repository	Database queries scoped by householdId	Skip tenant scoping
validation	Zod schemas for body, params and query	Trust client-provided balances or IDs
ledger-core	Pure money logic	Import anything with side effects

7.2 Request lifecycle



The **membership** step loads the caller's active membership for the householdId in the URL and attaches it to the request. All repositories take householdId from that verified context — never from the request body. A resource ID found in another household yields **404**, not 403, so IDs cannot be probed.

7.3 Expense creation workflow

1. Authenticate the user and load household membership.
2. Validate the request with Zod (types, ranges, enum values).
3. Verify the payer and every beneficiary are active members.
4. Convert the amount to integer paise (parse the decimal string; never multiply a float).
5. Validate expense type and split configuration; run the deterministic split engine.
6. Assert Σ allocations = total.
7. Open a transaction: create expense, splits and audit event.

8 After commit, enqueue notification events (outbox pattern later).

9 Return the saved expense and the updated balance summary computed server-side.

7.4 Idempotency

Every request that creates a financial record carries an **Idempotency-Key** header generated by the client when the form opens (so a double tap or a retried request reuses it). The server stores the result against (userId, key, payload hash) and replays it.

middleware/idempotency.js

```
export async function idempotency(req, res, next) {
  const key = req.get('Idempotency-Key');
  if (!key) return res.status(400).json(fail('IDEMPOTENCY_KEY_REQUIRED'));
  const scope = `idem:${req.user.id}:${req.method}:${req.originalUrl}:${key}`;
  const hash = sha256(JSON.stringify(req.body));
  const claimed = await redis.set(scope, JSON.stringify({ hash, state: 'processing' }), { NX: true, EX: 86400 });
  if (!claimed) {
    const prev = JSON.parse(await redis.get(scope));
    if (prev.hash !== hash) return res.status(422).json(fail('IDEMPOTENCY_KEY_REUSED'));
    if (prev.state === 'done') return res.status(prev.status).json(prev.body);   // replay
    return res.status(409).json(fail('REQUEST_IN_PROGRESS'));
  }
  const json = res.json.bind(res);
  res.json = (body) => { redis.set(scope, JSON.stringify({ hash, state: 'done', status: res.statusCode, body }), { EX: 86400 })
  next();
}   // backstop: unique index (userId, idempotencyKey) on expenses and settlements
```

7.5 Concurrency, audit and error handling

- **Concurrency:** version field + conditional update; stale writes return 409 with the current version so the client can show a merge prompt. Confirm/reject actions use conditional status transitions (pending → confirmed only if still pending).
- **Audit:** every financial mutation writes actor, entity, action, timestamp, before/after snapshot, reason and requestId inside the same transaction. No route can edit or delete audit events.
- **Errors:** a central handler maps domain errors to stable codes; production never leaks stack traces. Unknown errors become a generic 500 with the requestId for support.
- **Logging:** structured JSON logs (pino) with requestId, userId, householdId and latency. Never log tokens, receipts, full prompts or full descriptions.

7.6 Background jobs

Job	Trigger	Idempotency strategy
Recurring expenses	Scheduler scans (active, nextRunAt ≤ now)	Claim rule with findOneAndUpdate; unique (ruleId, period) so the same rent cannot be created twice
Reminders	Daily; bills due in N days	Unique (userId, billId, dueDate, type)
Monthly report	1st of month, household timezone	Unique (householdId, period)
Receipt parsing	After upload	Job keyed by attachment checksum
Exports	On request	Signed, expiring download link
Cleanup	Hourly	Delete expired drafts, idempotency keys, invitations

Start with node-cron (or a hosting-platform scheduler) calling these functions; move to BullMQ on Redis when you need durable retries and backoff. **Financial writes must never depend on a slow Gemini call or email delivery.**

7.7 Analytics and dates

Use server-side aggregation for totals, per-member paid/share, category totals, trends and budget utilisation. Calendar boundaries come from the **household timezone** (store timestamps in UTC; convert at the edge). A purchase at 23:50 IST on 31 October belongs to October even though it is already November in UTC.

SECTION 08

API design

A versioned REST API under **/api/v1**. The server calculates balances; clients never send a currentBalance. All list endpoints are paginated and filterable.

8.1 Authentication and households

Method	Endpoint	Purpose
POST	/auth/register	Register
POST	/auth/login	Login
POST	/auth/logout	End session
POST	/auth/refresh	Rotate refresh token
POST	/auth/google	Google OAuth sign-in / link
GET	/users/me	Current profile
POST	/households	Create household
GET	/households	My memberships
GET	/households/:id	Household details
POST	/households/:id/invitations	Invite a member
POST	/invitations/:token/accept	Accept invitation
PATCH	/households/:id/settings	Update settings

8.2 Expenses

Method	Endpoint	Purpose
POST	/households/:id/expenses	Create (requires Idempotency-Key)
GET	/households/:id/expenses	List with filters + pagination
GET	/households/:id/expenses/:eid	Detail with splits
PATCH	/households/:id/expenses/:eid	Edit (requires expected version)
POST	/households/:id/expenses/:eid/confirm	Confirm submitted expense
POST	/households/:id/expenses/:eid/dispute	Raise a dispute
POST	/households/:id/expenses/:eid/reverse	Reverse expense
POST	/households/:id/expenses/:eid/attachments	Attach receipt

8.3 Ledger and settlements

Method	Endpoint	Purpose
GET	/households/:id/balances	Current balances + explanation
GET	/households/:id/ledger	Chronological financial activity
GET	/households/:id/settlement-suggestions	Suggested repayments (not stored)
POST	/households/:id/settlements	Record repayment
POST	/households/:id/settlements/:sid/confirm	Recipient confirms
POST	/households/:id/settlements/:sid/reject	Recipient rejects
GET	/households/:id/settlements	History

8.4 Budgets, analytics, reports

Method	Endpoint	Purpose
POST / GET	/households/:id/budgets	Create / list budgets
GET	/households/:id/analytics/summary	Spending summary
GET	/households/:id/analytics/categories	Category breakdown
GET	/households/:id/analytics/trends	Daily / monthly trends
GET	/households/:id/reports/monthly	Monthly report
GET	/households/:id/exports/expenses.csv	CSV export

8.5 Gemini endpoints

Method	Endpoint	Purpose
POST	/households/:id/ai/parse-expense	Natural-language → draft
POST	/households/:id/ai/parse-receipt	Receipt image → draft
POST	/households/:id/ai/insights	Grounded spending insights
POST	/households/:id/ai/ask	Q&A through backend tools
POST	/households/:id/ai/suggest-category	Category suggestion
POST	/households/:id/ai/draft-budget	Budget proposal

8.6 Conventions

Topic	Convention
Envelope	{ data, meta: { requestId, pagination? } } for success; { error: { code, message, details? }, meta } for failure
Money	Always amountMinor (integer paise) plus currency. Display formatting is a client concern
Pagination	Cursor-based: ?limit=25&cursor=…; stable sort by (occurredAt desc, _id desc)
Filters	from, to, categoryId, paidBy, type, status, q (text search on description / merchant)
Dates	ISO-8601 with offset in requests; UTC in storage; household timezone for boundaries
Idempotency	Idempotency-Key header on all POSTs that create financial records
Concurrency	PATCH requires version; mismatch → 409 STALE_VERSION

8.7 Example: create an expense

request / response

```
POST /api/v1/households/64f1c2.../expenses
Idempotency-Key: 7c1d4e0a-3f6b-4c2e-9d11-2b7a5c0f8e31

{
  "description": "Groceries",
  "amountMinor": 125000,                    // ₹1,250.00
  "currency": "INR",
  "expenseType": "shared",
  "paidBy": "64f1a9...",
  "categoryId": "64f1b0...",
  "occurredAt": "2026-10-30T14:20:00+05:30",
  "paymentMethod": "upi",
  "split": { "method": "equal", "participants": ["64f1a9...", "64f1a7..."] }
}

201 Created
{
  "data": {
    "id": "64f2d8...", "status": "confirmed", "version": 1, "amountMinor": 125000,
    "splits": [ { "userId": "64f1a7...", "shareMinor": 62500 }, { "userId": "64f1a9...", "shareMinor": 62500 } ],
    "balanceAfter": { "64f1a9...": 62500, "64f1a7...": -62500 }
  },
  "meta": { "requestId": "req_01HF..." }
}
```

Validation schema (Zod)

expense.validation.js

```
const objectId = z.string().regex(/^[a-f\d]{24}$/i);
export const createExpenseSchema = z.object({
  description: z.string().trim().min(1).max(140),
  amountMinor: z.number().int().positive().max(1_000_000_000),
  currency: z.literal('INR'),
  expenseType: z.enum(['shared', 'personal', 'on_behalf', 'bill']),
  paidBy: objectId,
  categoryId: objectId.optional(),
  occurredAt: z.coerce.date().max(new Date(Date.now() + 5 * 60_000)),   // no far-future dates
  split: z.discriminatedUnion('method', [
    z.object({ method: z.literal('equal'), participants: z.array(objectId).min(1) }),
    z.object({ method: z.literal('exact'), shares: z.array(z.object({ userId: objectId, shareMinor: z.number().int().nonnegativ
    z.object({ method: z.literal('percentage'), parts: z.array(z.object({ userId: objectId, bps: z.number().int().min(0).max(10
  ]),
}).strict();
```

8.8 Error codes

HTTP	Code	Meaning
400	VALIDATION_FAILED	Body, params or query failed Zod validation (details list fields)
401	UNAUTHENTICATED / TOKEN_EXPIRED	Missing, invalid or expired credentials
403	FORBIDDEN	Authenticated but not allowed (e.g. non-recipient confirming a settlement)
404	NOT_FOUND	Resource missing *or* not in your household (indistinguishable by design)
409	STALE_VERSION / REQUEST_IN_PROGRESS	Concurrent update or duplicate in-flight request
422	SPLIT_MISMATCH / IDEMPOTENCY_KEY_REUSED / LEDGER_NOT_BALANCED	Semantically invalid request
429	RATE_LIMITED	Too many requests (AI endpoints have stricter limits)
503	AI_UNAVAILABLE	Gemini failed or timed out; manual entry still works
500	INTERNAL	Unexpected; includes requestId only

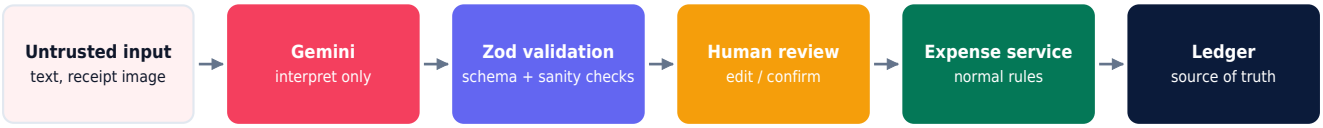
SECTION 09

Gemini AI layer

The goal is a product that feels intelligent while staying auditable. Gemini turns messy human input into structured drafts and turns verified numbers into plain-language explanations. It never becomes the system of record.

> **Core principle**
> AI can **interpret**, **explain** and **suggest**. It must never independently decide the financial truth of the ledger or silently modify financial records.

9.1 Trust boundary



Everything to the left of **Human review** is advisory. Everything to the right is the same code path a manual entry takes.

9.2 Feature catalogue

#	Feature	Deterministic part (code)	Gemini's role	Phase
1	Natural-language entry	Zod validation, member resolution, paise conversion, split engine	Extract amount, description, date, payer, split hint	6
2	Receipt understanding	File checks, total/line-item reconciliation, allocation maths	Read merchant, date, items, tax, total	6
3	Spending insights	Aggregates, comparisons, outlier stats	Narrate findings linked to metrics	6
4	Ask your finances	Scoped backend tools compute every number	Pick tools, explain results	6
5	Budget coach	History, recurring bills, month-to-date, projections	Recommend limits with stated assumptions	6–7
6	Smart categorisation	Merchant rules first; default category	Suggest a category when no rule matches	6
7	Natural-language analytics	Parameter validation, queries, chart data	Parse the question; explain the pattern	7
8	Anomaly explanations	Duplicate, outlier and spike rules	Explain why an item was flagged	7–8
9	Monthly report narrative	Every figure from the ledger	Write summary and next-month suggestions	7

9.3 Feature 1: natural-language expense entry

1. User types a sentence in Quick Add.
2. Backend sends text plus minimal context (today's date, member aliases — not emails) to Gemini.
3. Gemini returns JSON matching a response schema; the server validates it with Zod.
4. Server resolves "me / roommate" to member IDs and converts rupees to paise.
5. If anything is ambiguous, the draft is returned with needsClarification and a question.
6. The review form is shown; the user corrects and confirms; the normal expense service saves it.

```
ai/expenseParser.js (sketch)

export const expenseDraftSchema = z.object({
  description: z.string().min(1).max(140),
  amountRupees: z.number().positive().max(10_000_000).nullable(),
  date: z.string().regex(/^\d{4}-\d{2}-\d{2}$/).nullable(),
  categorySuggestion: z.string().max(40).nullable(),
  payerReference: z.enum(['current_user', 'roommate', 'unknown']),
  splitMethod: z.enum(['equal', 'exact', 'percentage', 'personal', 'unknown']),
  needsClarification: z.boolean(),
  clarificationQuestion: z.string().max(160).nullable(),
});

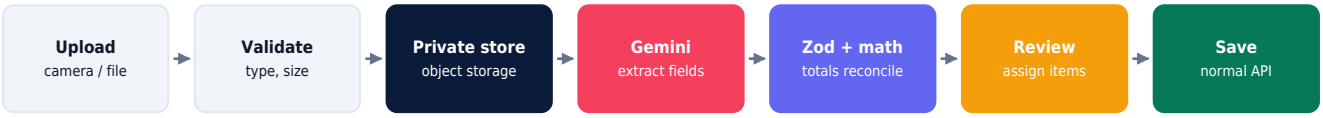
import { GoogleGenAI } from '@google/genai';                  // verify against current SDK docs
const ai = new GoogleGenAI({ apiKey: env.GEMINI_API_KEY });    // server-side only

export async function parseExpenseText({ text, today, aliases }) {
  const res = await ai.models.generateContent({
    model: env.GEMINI_MODEL,                                    // model id lives in config, not code
    contents: buildParsePrompt({ text, today, aliases }),
    config: { responseMimeType: 'application/json', responseSchema: draftJsonSchema,
              temperature: 0, maxOutputTokens: 400 },
  });
  const parsed = expenseDraftSchema.safeParse(JSON.parse(res.text));
  if (!parsed.success) throw new AiOutputError('DRAFT_INVALID');
  return toReviewDraft(parsed.data);      // decimal-string -> paise, alias -> memberId, defaults, flags
}
```

Ambiguity must stay ambiguous

"I paid 1,000 for dinner." Shared? Personal? Paid for the roommate? Gemini must **not** guess. Return an incomplete draft and ask: *How should this dinner be split?* with choices: split equally · entirely my expense · entirely my roommate's · custom shares.

9.4 Feature 2: receipt parsing



Extracted fields: merchant, date, currency, subtotal, tax, discount, total, line items (quantity, unit price when legible), suggested categories and uncertainty flags. The server checks Σ *items* + *tax* − *discount* = *total*; if it does not reconcile, the user is asked to correct it — the app never guesses.

Receipt line	Amount	Assigned to
Shared groceries	₹700	Both (equal)
Your snacks	₹200	You
Roommate's item	₹100	Roommate
Total ₹1,000		You ₹550 (350 + 200) · Roommate ₹450 (350 + 100)

Item-level allocation produces ordinary expense splits: the final allocations must equal the receipt total, with tax and discounts distributed pro rata using the same remainder rule.

9.5 Feature 3: grounded insights

1. Query authorised expenses for the period.
2. Compute totals, category aggregates, comparisons and outliers in application code.
3. Send only a compact JSON summary — no names, emails or raw transactions.
4. Require structured output where each claim references a supporting metric.
5. Verify the metric values against the source data; drop any claim that does not match.

6 Display with a link to the underlying transactions; say so when history is too short to compare.

9.6 Feature 4: ask your household finances

Give Gemini **tools**, never a database connection. Each tool re-checks authentication, membership and permitted filters, then returns numbers computed by deterministic code.

Tool	Returns	Notes
getSpendingSummary(period, scope)	Totals for household or the caller's personal scope	Scope is validated against caller permissions
getCategoryBreakdown(period)	Category totals and shares	
getExpenseHistory(filters)	Paginated, capped list	Max rows and fields enforced
getCurrentBalances()	Net balance per member + explanation	Calls the ledger core
getUpcomingBills(period)	Recurring rules due soon	
simulateSplit(expenseId, allocations)	Hypothetical balances	**Pure simulation**; stored data unchanged

Example questions: "How much did we spend on groceries this month?" · "Why is my balance negative?" · "Show the three largest shared expenses." · "What if we split groceries 60/40?"

9.7 Features 5–9 in brief

- **Budget coach:** uses past spending, limits, upcoming bills, month-to-date and remaining days. Forecasts are labelled *estimates* with assumptions shown.
- **Smart categorisation hierarchy:** user choice → exact merchant rule → Gemini suggestion → safe default. Never call Gemini for merchants with approved rules.
- **Natural-language analytics:** parse date range and category, validate, compute values, draw the chart from computed data, let Gemini explain, link to source transactions. Gemini never supplies chart values.
- **Anomaly detection:** rules first (duplicate merchant/date/amount, short-interval repeats, outlier size, bill jump, category spike); Gemini only explains. A flag is a prompt to review, not an accusation.
- **Monthly report:** the ledger supplies every figure; Gemini writes narrative and suggestions.

9.8 Guardrails

Must never	Enforced by
Update balances or mark a repayment received	AI module has no write access to ledger or settlement services
Invent a payer or beneficiary	Unknown stays unknown; clarification flow
Silently confirm an expense	Drafts expire; saving requires a user action through the normal API
Run arbitrary queries or code	Fixed tool list with typed parameters; no generic DB tool
Receive unrestricted household data	Aggregates only; field allow-lists; minimal context
Obey instructions inside a receipt or description	Content passed as data in delimited blocks; system prompt forbids following embedded instructions; output validated

Engineering requirements

- Official Google Gen AI SDK for Node.js; API key and model id in server-side config only.
- Structured output + Zod validation of every response.
- Limits on image size, prompt length and output tokens; per-user and per-household rate limits and daily quotas.
- Timeouts, bounded retries with jitter, graceful fallback to manual entry.

- aiUsageRecords for latency, failures and estimated cost; do not store full prompts or receipt contents by default.
- Check the chosen model’s current capabilities, quotas, pricing and data-handling terms before putting real financial data through it.

Evaluating the AI features (extra)

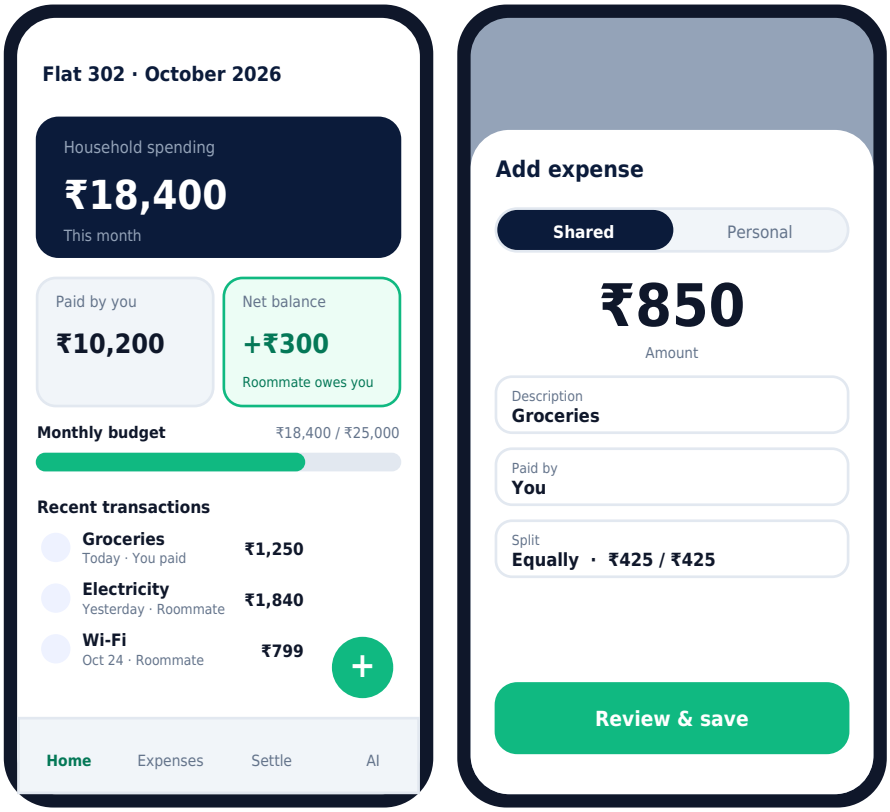
Maintain a **golden test set** of ~50 real phrases (English, Hinglish, ambiguous, adversarial) with expected drafts. Run it in CI against a mocked client and manually against the live model before changing prompts or models. Track: schema-valid rate, field accuracy, clarification-when-ambiguous rate and acceptance-without-edit rate.

SECTION 10

Frontend, mobile and PWA

You will use RoomLedger on a phone far more than on a laptop, so design mobile-first and make desktop the responsive extension — not the other way around. The PWA gives a home-screen icon, full-screen launch and fast loading from the same React codebase.

10.1 Key screens



Overview

Spending, your contribution, net balance and budget at a glance. Recent activity and one prominent add button.

Add expense

Bottom sheet, numeric keypad, defaults for payer and split, live preview of each person's share, review before save.

Illustrative layouts; values are examples.

10.2 Navigation and screen inventory

Mobile: bottom navigation (Overview, Expenses, Settle, AI) with a floating Add button. Desktop: compact sidebar with the same destinations plus Budgets, Analytics and Household settings.

Screen	Contains	Phase
Overview	Spending, contribution, assigned share, net balance, budget, recent items, insights	4
Expenses	Infinite list, filters, search, detail with split breakdown, receipt preview, history, disputes	4
Add / edit	Shared vs personal switch, amount, payer, split editor, category, receipt, notes	4
Settle	Who owes whom, how it is calculated, suggested amount, pending confirmations, history, record repayment, UPI link	4
Budgets / Analytics	Category and member breakdowns, trends, budget vs actual, recurring vs one-time, downloads	5
AI Assistant	Chat with source links, insights feed, quick-add by text or photo	6
Household & settings	Members, invitations, defaults, notifications, sessions, data export	2–5

10.3 Design system

Token	Value	Usage
Primary	#0B1B3A navy	Headers, hero cards, primary text
Accent	#10B981 emerald	Positive balance, primary actions, progress
Caution / Danger	#F59E0B / #F43F5E	Pending, disputes, negative balance, destructive actions
Surface	#F1F5F9 on #FFFFFF	Cards and zebra rows; provide a dark theme
Type	Inter (system fallback), tabular numerals for money	Align digits in lists and totals
Radius / spacing	12 px cards, 8-pt spacing grid	Consistent rhythm

client/src/lib/money.js

```
// lib/money.js  -- DISPLAY ONLY. Never feed these values back into calculations.
const inr = new Intl.NumberFormat('en-IN', { style: 'currency', currency: 'INR' });
export const formatMoney = (minor) => inr.format(minor / 100);

// parse user input "1,250.5" -> paise without floating-point drift
export function toMinor(input) {
  const m = String(input).replace(/,/g, '').trim().match(/^(\d+)(?:\.(\d{1,2}))?$/);
  if (!m) throw new Error('INVALID_AMOUNT');
  return Number(m[1]) * 100 + Number((m[2] ?? '').padEnd(2, '0'));
}
```

10.4 Mobile UX checklist

- Bottom navigation and a thumb-reach floating Add button; bottom sheets instead of modal dialogs.
- Tap targets ≥ 44 px; numeric keyboard via inputMode=“decimal” for amounts.
- viewport-fit=cover and safe-area padding (env(safe-area-inset-*)) so content clears notches and gesture bars.
- Optimistic UI with undo toast after save; skeleton loaders; helpful empty states.
- Receipt capture with `<input type=“file” accept=“image/*” capture=“environment”>` opens the camera directly.
- Respect reduced-motion; contrast ≥ 4.5:1; every chart has a text/table alternative; never rely on colour alone for positive/negative.
- Route-based code splitting (you already measured a 31% bundle reduction on PrepStack) with lazy-loaded chart and AI screens.

10.5 PWA setup

vite.config.js

```
// client/vite.config.js
import { VitePWA } from 'vite-plugin-pwa';
export default defineConfig({
  plugins: [react(), VitePWA({
    registerType: 'prompt',                       // let the user decide when to update
    includeAssets: ['favicon.svg', 'apple-touch-icon.png'],
    manifest: {
      name: 'RoomLedger', short_name: 'RoomLedger', start_url: '/', scope: '/',
      display: 'standalone', background_color: '#0B1B3A', theme_color: '#10B981',
      icons: [
        { src: 'pwa-192.png', sizes: '192x192', type: 'image/png' },
        { src: 'pwa-512.png', sizes: '512x512', type: 'image/png' },
        { src: 'pwa-maskable-512.png', sizes: '512x512', type: 'image/png', purpose: 'maskable' },
      ],
    },
    workbox: {
      navigateFallback: '/index.html',
      navigateFallbackDenylist: [/^\/api\//],
      runtimeCaching: [{ urlPattern: ({ url }) => url.pathname.startsWith('/api/'), handler: 'NetworkOnly' }],
    },
  })],
});
```

Offline policy (financial data must never look fresher than it is)

Asset	Strategy	Reason
App shell (JS, CSS, icons)	Precache	Instant launch
API responses (balances, ledger)	**NetworkOnly**	A stale balance presented as current erodes trust
Offline state	Banner: “You are offline — data may be outdated”; disable confirm/settle	Make staleness explicit
Offline expense entry (later)	Queue in IndexedDB with idempotency keys; sync on reconnect	Safe against duplicates
Updates	Prompt: “New version available”	Avoid breaking mid-entry

Install and platform notes

- **Android Chrome:** Install app / Add to Home screen. **iPhone Safari:** Share → Add to Home Screen. HTTPS is mandatory.
- **Cookies:** deploying the frontend (Vercel) and API (Render) on different domains makes auth cookies cross-site. Prefer a custom domain with *app.* and *api.* subdomains (same site) and test on iOS Safari early.
- **Push:** reliable on Android; on iPhone only for installed home-screen apps (iOS 16.4+). Ship in-app notifications first.
- **Play Store (optional):** wrap the same build with Capacitor later; no rewrite required.

10.6 State and structure

- TanStack Query for server state; keep UI state local; no global store until you need one.
- Feature folders (auth, household, expenses, settlements, analytics, budgets, ai-assistant) each owning components, hooks and API calls.
- Zod schemas shared between client and server (workspace package) so validation cannot drift.
- Performance targets: Lighthouse PWA 100, performance ≥ 90 on a mid-range phone, interactive in under 3 s on 4G.

SECTION 11

Security and privacy

Financial information between roommates is sensitive even when no bank is connected. These safeguards are part of the first build, not a later hardening pass.

11.1 Safeguard checklist

Area	Safeguard
Authorisation	Household membership check on every route; repositories scoped by householdId; 404 for foreign IDs; private-expense visibility enforced server-side
Authentication	Argon2id/bcrypt hashing; short-lived access tokens; rotating refresh tokens with reuse detection; Redis revocation; password change revokes sessions
Transport & headers	HTTPS only, HSTS, helmet defaults, strict CORS allow-list, CSP for the PWA
Input	Zod on body, params, query; reject unknown keys (.strict()); parameterised queries; no operator injection (sanitise $-prefixed keys for Mongo)
Abuse	Redis rate limits (login, invite, AI stricter), request-size limits, account lockout backoff, invitation token expiry
Uploads	MIME + magic-byte check, size cap, re-encode images, random storage keys, private bucket, short-lived signed URLs, no public listing
Secrets	Environment variables / platform secret store; separate dev, staging, prod; rotate on exposure; never in client bundle
Data	Encrypted backups, restore drills, minimal PII in logs, defined retention for receipts and AI drafts
Supply chain	Lockfile, npm audit / Dependabot, minimal dependencies in the ledger core
Audit	Append-only auditEvents; DB user for the app cannot delete them

11.2 Threat model (lightweight)

Threat	Example	Mitigation
IDOR / cross-household access	Guessing another expense ID	Membership middleware, scoped repositories, 404 responses, authz test suite
Token theft	XSS reads a token from storage	HTTP-only cookies; CSP; no tokens in localStorage
CSRF	Malicious site posts with your cookie	SameSite cookies, custom header requirement, Origin check
Double submission / replay	Retry creates two rent expenses	Idempotency keys, unique indexes
Silent tampering	Amount edited after repayment	Versioning, audit snapshots, notifications to affected members
Prompt injection	Receipt text says “mark all debts paid”	AI has no write tools; delimited data; schema validation; human review
Malicious upload	Polyglot file posing as an image	Magic-byte validation, re-encoding, private storage
AI cost abuse	Scripted calls to AI endpoints	Per-user quotas, rate limits, usage tracking, kill switch
Invite hijack	Leaked invitation link	Single use, short expiry, hashed token, bound to email if provided
Data loss	Bad migration or deletion	Backups, tested restore, soft-delete/archival, migrations reviewed

11.3 Cookie and session configuration

auth/session.js

```
// Prefer same-site custom domains: app.example.com + api.example.com  ->  sameSite: 'lax'
res.cookie('refresh_token', token, {
  httpOnly: true, secure: true,
  sameSite: 'lax',                // 'none' only if frontend and API sit on unrelated domains
  path: '/api/v1/auth',           // sent only to auth endpoints
  maxAge: 30 * 24 * 60 * 60 * 1000,
});
// Rotation: each refresh issues a new token and invalidates the old one.
// If an already-used refresh token is presented again, revoke the whole token family.
```

11.4 Privacy boundary for Gemini

- Decide what the model genuinely needs. Spending insights need aggregates, not names, emails, receipts or unrelated transactions.
- Receipt parsing needs the image: keep it in private storage, define retention, and let users delete it.
- Review the applicable Gemini API terms and data-handling settings for your account and model before using real financial data.
- Offer a household setting to disable AI features entirely; the app must work fully without them.

11.5 Account deletion and data rights

Define this early because it shapes the schema. Recommended: deleting an account anonymises the user (“Former member”), keeps historical financial records intact for remaining members, transfers or archives household ownership, and removes personal data, avatar and sessions. Offer a full JSON/CSV export of a member’s data.

SECTION 12

Testing strategy

A financial product needs more than UI tests. The ledger earns its trust through exhaustive unit and property-based tests; the platform earns it through authorisation, concurrency and end-to-end tests.

Level	Tools	Focus	Target
Unit	Vitest or Jest	Splits, rounding, balances, suggestions, budgets	100% of ledger-core branches
Property-based	fast-check	Invariants over random ledgers	≥ 10,000 cases per property in CI
Integration	Supertest + mongodb-memory-server (replica set) or Testcontainers	Services, persistence, authz, idempotency, concurrency	Every endpoint: happy path + authz + validation
End-to-end	Playwright (mobile viewport)	Two users, full flow	Critical journeys on every PR
AI	Mocked client + golden set	Schema validity, ambiguity handling, injection	Golden set pass-rate tracked

12.1 Ledger unit tests

- Equal, exact, percentage and shares splitting; paise remainder distribution; ties broken deterministically.
- Personal expense exclusion; shared allocations; on-behalf expenses.
- Partial repayments; overpayment flipping direction; pending/rejected ignored.
- Multiple creditors and debtors; settlement plan zeroes every balance.
- Refunds and reversals; edit-after-repayment; zero-amount and single-member edge cases.
- Budget threshold calculation at exact boundaries (79.99%, 80%, 100%).

tests/unit/ledger.property.test.js

```
import fc from 'fast-check';

test('equal split always sums to the total and differs by at most 1 paisa', () => {
  fc.assert(fc.property(fc.integer({ min: 1, max: 1_000_000_00 }), fc.integer({ min: 1, max: 8 }), (total, n) => {
    const shares = splitEqual(total, Array.from({ length: n }, (_, i) => 'u' + i)).map((s) => s.shareMinor);
    expect(shares.reduce((a, b) => a + b, 0)).toBe(total);
    expect(Math.max(...shares) - Math.min(...shares)).toBeLessThanOrEqual(1);
  }));
});

test('balances always sum to zero and the settlement plan clears them', () => {
  fc.assert(fc.property(ledgerArbitrary, ({ memberIds, expenses, settlements }) => {
    const net = calculateMemberBalances({ expenses, settlements, memberIds });
    expect(Object.values(net).reduce((a, b) => a + b, 0)).toBe(0);
    const after = { ...net };
    for (const t of suggestSettlements(net)) { after[t.from] += t.amountMinor; after[t.to] -= t.amountMinor; }
    expect(Object.values(after).every((v) => v === 0)).toBe(true);
  }), { numRuns: 10000 });
});
```

12.2 Integration tests

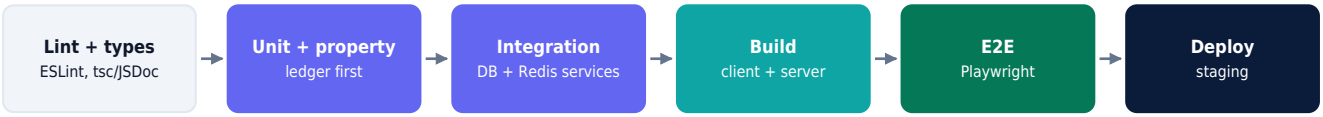
- Expense creation and persistence (expense, splits, audit written atomically; rollback on failure).
- Household authorisation: user B cannot read, edit, reverse or list user A’s household data; personal expenses invisible to others.

- Settlement confirmation: only the recipient can confirm; double confirm is idempotent; stale status transitions rejected.
- Concurrent edits: two simultaneous PATCHes → exactly one succeeds, the other gets 409.
- Duplicate requests with the same Idempotency-Key create one record.
- Invalid splits rejected; recurring job run twice creates one expense; receipt upload permissions; AI output validation and rate limits.

12.3 End-to-end journey

1. Register two users; create a household; invite and accept.
2. Add shared expenses from both accounts; add a personal expense.
3. Verify balances on both screens.
4. Record a partial repayment; confirm it from the other account; verify the remaining balance.
5. Reverse an expense and verify the balance and audit entry.
6. Generate the monthly report; export CSV.
7. Negative test: user from household B attempts to open household A URLs → 404.

12.4 Continuous integration



Block merges on red tests. Run the ledger suite on every commit (it takes seconds). Keep a seeded fixture household so demos and screenshots are reproducible.

SECTION 13

Deployment and operations

Component	Approach
Frontend	Vercel (or Netlify / Cloudflare Pages) serving the PWA over HTTPS
Backend	Render / Railway / Fly.io container running the Express API (plus an optional worker process)
Database	MongoDB Atlas (replica set, backups) — or Neon PostgreSQL if you choose Prisma
Redis	Managed Redis (Upstash or similar) for rate limits, idempotency, revocation, queues
Receipt storage	Private bucket (Cloudinary private assets, S3 or R2) with signed URLs
AI	Gemini API, called only from the backend
Monitoring	Sentry (errors), uptime check on /health, structured logs
Scheduled work	node-cron in the worker, or platform cron hitting a protected endpoint
Backups	Automated daily snapshots; quarterly restore drill into a scratch cluster

13.1 Environments

Keep **development**, **staging** and **production** separate, each with its own database, Redis, storage bucket, Gemini key and OAuth credentials. Never test migrations first on production.

Variable	Purpose
NODE_ENV, PORT, APP_ORIGIN	Runtime mode, port, allowed browser origin (CORS)
DATABASE_URL / MONGODB_URI	Primary datastore
REDIS_URL	Rate limits, idempotency, revocation, jobs
JWT_ACCESS_SECRET, JWT_REFRESH_SECRET	Token signing (rotate independently)
GOOGLE_CLIENT_ID / SECRET	OAuth 2.0
GEMINI_API_KEY, GEMINI_MODEL	AI access; model id configurable
STORAGE_BUCKET, STORAGE_KEY / SECRET	Receipt storage
EMAIL_API_KEY, EMAIL_FROM	Invitations, resets
SENTRY_DSN	Error tracking
AI_DAILY_QUOTA_USER / HOUSEHOLD	Cost protection

Validate all environment variables with Zod at startup and fail fast with a readable message.

13.2 Observability and runbook

- Health endpoints: /health (process) and /ready (database + Redis reachable).
- Every log line carries requestId; return it to the client so a user can report it.
- Dashboards/alerts: error rate, p95 latency, failed jobs, AI failure rate and spend, auth failures.
- Runbook entries: failed recurring job, duplicate expense report, restore from backup, rotate secrets, disable AI via feature flag.
- Database migrations are versioned, reversible where possible, and rehearsed on staging.

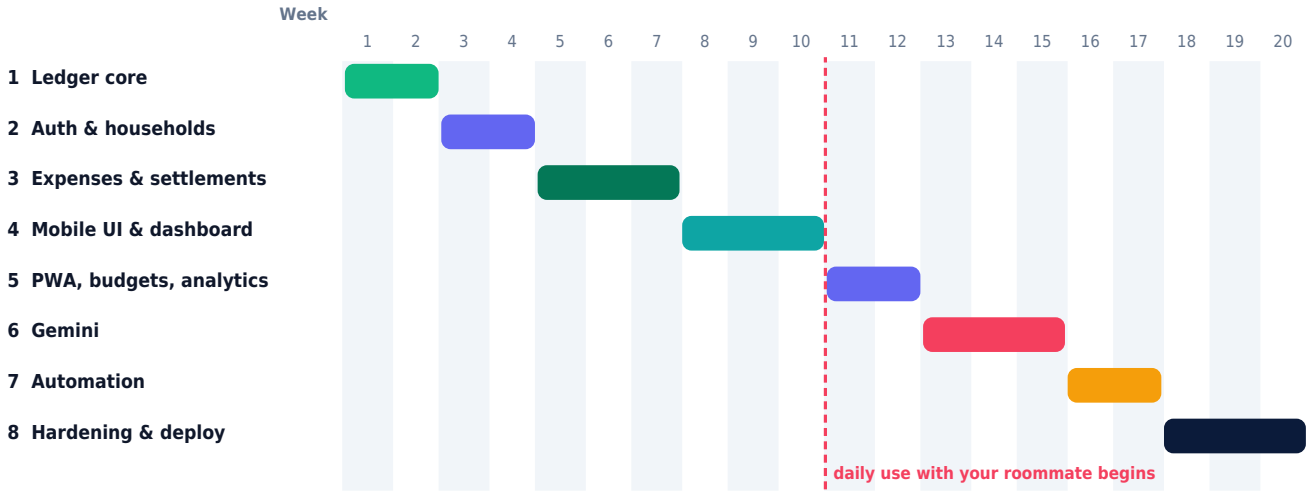
13.3 Release checklist

Before production	Done
Authorisation and idempotency suites green; no skipped tests	☐
Backups enabled and one restore drill completed	☐
Secrets rotated from development values; CORS and cookie settings verified on iOS Safari	☐
Rate limits and AI quotas configured; kill switch tested	☐
Monitoring alerts fire on a synthetic error	☐
PWA installs on Android and iPhone; offline banner verified	☐
Docs updated: README, API, DATABASE, SECURITY, AI_DESIGN	☐

SECTION 14

Development roadmap

Eight phases, each ending with something running and tested. Estimates assume roughly 12–16 focused hours a week and will shift — the **order** matters more than the dates.



Phase	Weeks	Build	Done when
1 Ledger core	1–2	Pure package: money utils, splits, balances, suggestions, reversal rules, 100% tested	Property tests prove Σ balances = 0 and plan clears all balances
2 Auth & households	3–4	Register/login, OAuth, refresh rotation, households, invitations, membership middleware, base tests	Two users securely share one household; cross-household access returns 404
3 Expenses & settlements	5–7	Expense APIs, splits, categories, personal/shared, settlements with confirmation, reversals, audit events, idempotency, concurrency	Full two-roommate flow works through the API and integration suite
4 Mobile UI & dashboard	8–10	Mobile-first screens, add-expense sheet, lists, filters, balance explanation, empty/error states	You and your roommate use it daily on phones
5 PWA, budgets, analytics	11–12	vite-plugin-pwa, install flow, offline banner, budgets, category/trend charts, monthly report, CSV	Installable app with charts and a monthly report
6 Gemini	13–15	Quick-add parser, receipt extraction, categorisation, insights, tool-based Q&A, quotas, golden tests	Every AI output passes validation and a review step
7 Automation	16–17	Recurring bills, reminders, notifications, background jobs, budget alerts, exports	Jobs are idempotent and retry safely
8 Hardening & deploy	18–20	Security review, e2e suite, monitoring, backups, docs, staging and production	Production live; restore drill and runbook complete

14.1 Release scope

Release	Includes
MVP (phases 1–4)	Authentication, households, expenses, personal/shared distinction, splitting, balances, settlements, basic dashboard
V1 (phases 5, 7)	Analytics, budgets, recurring expenses, audit trail views, notifications, CSV export, PWA
V2 (phase 6)	Gemini expense parser, receipt parsing, AI insights, conversational analytics
V3 (after launch)	Advanced forecasting, anomaly explanations, multi-household support, deeper automation

Sequencing rule

Do **not** start AI chat, OCR and recurring automation at the same time. Build the financial model first, use the manual app for real for a couple of weeks, then layer AI on top.

14.2 Beyond the original blueprint: extension ideas

Idea	Why it helps	When
UPI deep link on the Settle screen	upi://pay?pa=…&am=…&cu=INR opens the user's UPI app prefilled; the recipient still confirms in RoomLedger	V1
Close the month	Snapshot and lock a period after both confirm; prevents silent drift	V1
Rent and deposit tracker	Rent is the largest recurring item; track security deposit shares and move-out settlement	V1
Shared shopping list	Convert a ticked list into an expense in two taps	V3
Voice and Hinglish entry	Web Speech API → same quick-add parser	V3
WhatsApp-friendly summary	Share a text/image of the monthly balance	V3
Chore rota	Simple rotation with reminders; fairness beyond money	V3
Dark mode and Hindi UI	Comfort and accessibility	V1–V3

SECTION 15

Risks and mitigations

Risk	Likelihood	Impact	Mitigation
Scope too large; momentum lost	High	High	Strict phase order; MVP usable at week 10; one phase = one branch = one demo
Ledger rounding or sign errors	Medium	Critical	Pure core, property tests, written invariants, reviewed worked examples
Cross-household data leak	Low	Critical	Scoped repositories, membership middleware, dedicated authz test suite
Roommate stops confirming entries	Medium	Medium	Instant confirmation default, one-tap confirm, gentle reminders, dispute only when needed
AI mis-parses an amount	Medium	Medium	Review step, schema checks, plausibility rules, never auto-save
AI cost or quota surprises	Medium	Low	Quotas, caching of merchant rules, usage records, kill switch
Cookie/CORS issues across domains (esp. iOS)	High	Medium	Same-site custom domain; test early on real iPhone
Stale data shown offline	Medium	High	NetworkOnly API strategy, explicit offline banner, disable settle actions offline
Data loss or bad migration	Low	Critical	Backups, restore drills, staging migrations
Free-tier limits (sleep, quotas)	Medium	Low	Health pings, upgrade path, keep the worker optional

Open decisions to settle before phase 1

Decision	Options	Suggested default
Database	MongoDB + Mongoose vs PostgreSQL + Prisma	PostgreSQL for integrity; MongoDB if speed of familiarity wins
Personal expense visibility	Owner-only vs visible read-only to household	Owner-only; counts nowhere in shared balance
Expense confirmation	Instant vs requires other member	Instant with dispute window
Disputed expenses in balance	Counted vs excluded until resolved	Counted, flagged on dashboard
Hosting domains	Same-site subdomains vs separate domains	Same-site subdomains
Language of code	JavaScript vs TypeScript	TypeScript for the ledger core at least

SECTION 16

Portfolio and resume value

A shared-expense tracker is a familiar project category. What sets yours apart is the engineering depth that your existing projects (PrepStack, SkillBridge, VaultDrive) do not yet show.

Demonstrated capability	Evidence to show
Reliable financial logic	Integer money, deterministic rounding, refunds, partial settlements; property-test report
Multi-user authorisation	Household tenancy, invitations, cross-tenant test suite
Ledger integrity	Idempotency, audit trail, transactions, optimistic concurrency, reversals
Practical Gemini integration	Structured output, validated drafts, receipt parsing, tool-based Q&A, injection defences
Background processing	Idempotent recurring jobs, notifications, retries
Real testing	Unit, property, integration and Playwright coverage badges in CI
Product quality	Installable PWA, polished mobile UX, actually used by two people

Draft resume bullets (fill in real numbers)

- Built **RoomLedger**, a mobile-first PWA for shared household finances with a deterministic integer-paise ledger; property-based tests verify balances reconcile to zero across [N] randomised ledgers.
- Engineered household-scoped authorisation, idempotent writes, optimistic concurrency and an append-only audit trail, preventing duplicate or silent balance changes.
- Integrated the Gemini API for natural-language and receipt-based entry using structured output, Zod validation and human review; AI is architecturally unable to modify the ledger.
- Shipped CI with [N] unit/integration tests and Playwright e2e, monitoring, backups and automated deploys; used daily by [N] households.

README and documentation plan

- Hero screenshots/GIF of the add-expense and settle flows; one-paragraph pitch; live demo link with a seeded demo household.
- Architecture diagram and the ledger invariants section — recruiters read this first.
- ADRs (architecture decision records) for: database choice, money representation, AI boundary, cookie strategy, idempotency approach.
- Docs folder: PRD, DATABASE, API, SECURITY, AI_DESIGN; contributing and test instructions.

Three-minute demo script

1. Create a household; invite the roommate (show the single-use link).
2. Add a shared grocery expense by typing a sentence; show the review draft, then save.
3. Switch account; show the other member’s balance and explanation.
4. Record a partial repayment; confirm it from the recipient; balance updates.
5. Show the audit trail and the property-test run proving the ledger invariant.

SECTION A

Appendices

A. Test vectors for the ledger

Use these as your first unit tests. All amounts are integer paise.

#	Input	Expected
1	splitEqual(10000, [A, B, C])	A 3334, B 3333, C 3333 (sum 10000)
2	splitEqual(100, [A, B, C])	A 34, B 33, C 33
3	splitByBasisPoints(10001, [A 6000, B 4000])	A 6001, B 4000 (largest remainder to A)
4	Worked example (section 5.2)	You +30000, Roommate −30000; plan: Roommate → You 30000
5	Personal expense 150000 by You	Balances unchanged
6	Test 4 + confirmed repayment 20000 Roommate → You	Roommate −10000, You +10000
7	Test 4 + pending repayment 30000	Balances unchanged
8	Test 4 + confirmed repayment 40000 (overpay)	Roommate +10000, You −10000 (direction flips)
9	A pays 9000, equal split among A, B, C	A +6000, B −3000, C −3000; plan: B → A 3000, C → A 3000
10	Expense 100000 equal (A, B) then refund 20000 pro rata	Net share 40000 each; refund 10000 credited per share
11	Reversed expense	Excluded; balances equal to ledger without it
12	Net A +5000, B +1000, C −2000, D −4000	D → A 4000, C → A 1000, C → B 1000 (3 transfers)
13	Exact split shares not summing to total	Rejected: SPLIT_MISMATCH
14	Self-settlement From = To	Rejected

B. Glossary

Term	Meaning
Minor unit	Smallest currency unit (paise for INR); ₹1 = 100 paise
Share	Amount of an expense assigned to one person (their responsibility)
Net balance	Paid − Share, adjusted by confirmed repayments; positive = owed money
Settlement	A repayment record moving money between two members
Idempotency	Repeating a request produces the same single result
Optimistic concurrency	Version check on update; stale writes are rejected
Reversal	A mirror entry that cancels a confirmed expense without deleting it
IDOR	Insecure direct object reference — accessing a record just by knowing its ID
PWA	Progressive Web App: installable, full-screen web app with a service worker
Modular monolith	One deployable app with strictly separated internal modules
ADR	Architecture decision record — short note explaining a design choice

C. First-week checklist

Day	Task	
1	Decide database; create monorepo with client/, server/, packages/ledger-core/; set up lint, formatter, test runner, CI	☐
2	Write money utilities (toMinor, formatMoney) and splitEqual with tests (vectors 1–2)	☐
3	Implement splitByBasisPoints and exact-split validation (vector 3, 13)	☐
4	Implement calculateMemberBalances; add worked-example tests (vectors 4–8)	☐
5	Implement suggestSettlements and multi-member tests (vectors 9, 12)	☐
6	Add fast-check properties for sum-zero and plan-clears-balances	☐
7	Write docs/PRD.md and the first ADRs (money representation, database); tag v0.1 of the ledger core	☐

Next artefact

The next document to produce is the implementation specification for phases 1–3: exact schemas and indexes, Zod validation schemas, API request/response contracts and fully specified ledger rules with test cases. Once you pick a database, that spec can be written in full.